

**PostgreSQL Production
Database Monitoring Kit**

Monitoring Using Native PostgreSQL Statistics

```
1
2  --PostgreSQL -- Production Database Monitoring Kit
3  /*
4  --The architecture becomes:
5
6  pg_stat_database
7      |
8      | every minute
9      ▼
10 database_snapshots
11     |
12     | calculate differences
13     ▼
14 database_delta
15     |
16     ├── Transaction throughput
17     ├── Rollback detector
18     ├── Cache hit ratio
19     ├── Database activity
20     ├── Session growth
21     ├── Abandoned sessions
22     ├── Fatal sessions
23     ├── Killed sessions
24     ├── Parallel worker pressure
25     ├── Temp file detector
26     ├── Database I/O latency
27     ├── Tuple activity
28     ├── Deadlock detector
29     ├── Checksum failure detector
30     └── Database workload ranking
31  */
32
```

Document Type	Technical SOP / Monitoring Runbook
Target Platform	PostgreSQL 18.x
Document Version	1.0
Prepared Date	26 August 2026
Classification	Technical / Internal Reference

Table of Contents

Document Control 3

1. Purpose and Monitoring Model 4

2. Scope and Important Corrections to the Picture 4

3. Prerequisites..... 4

4. Monitoring Schema Installation 5

5. Snapshot Collection Function..... 6

6. Delta View..... 7

7. Production Monitoring Queries..... 8

8. Accurate I/O Latency Monitoring 10

9. Parallel Worker Pressure 11

10. Current Session and Wait Monitoring..... 11

11. Optional SQL Workload Layer: pg_stat_statements 11

12. Scheduling the Snapshot 12

13. Retention and Housekeeping 12

14. Suggested Alert Conditions 12

15. Validation Procedure..... 13

16. Troubleshooting 13

17. Security and Least Privilege 13

18. Production Rollout Checklist 14

19. Complete Architecture Summary 14

20. Official PostgreSQL References 15

Appendix A - Deployment SQL Order 15

Appendix B - DBA Interpretation Rules 15

Document Control

Field	Value
Document Title	PostgreSQL Production Database Monitoring Kit
Purpose	Build a native PostgreSQL snapshot-and-delta monitoring framework for database-level production indicators.
Target Version	PostgreSQL 18.x
Primary Source Basis	Official PostgreSQL 18 documentation only for technical behavior and catalog/view definitions.
Reference Image	User-supplied architecture concept: pg_stat_database → snapshots → deltas → monitoring indicators.
Owner	PostgreSQL DBA / Database Engineering
Review Trigger	PostgreSQL major upgrade, monitoring schema change, or operational policy change.

Revision History

Version	Date	Change
1.0	26-Aug-2026	Initial controlled SOP.

Source and Accuracy Statement

The technical definitions, PostgreSQL catalog/view behavior, and SQL design in this SOP were prepared and cross-checked against the official PostgreSQL 18 documentation. The supplied image is used as the monitoring requirement and architecture concept, not as an authoritative PostgreSQL specification.

Accuracy wording: A literal “100% accuracy” guarantee is not technically defensible for executable operational material because results depend on PostgreSQL minor release, configuration, permissions, workload, statistics resets, and operating environment. This document therefore states the stronger verifiable claim: technical PostgreSQL behavior is sourced exclusively from the cited official PostgreSQL documentation and the scripts are designed for PostgreSQL 18.x.

1. Purpose and Monitoring Model

The goal is to convert PostgreSQL cumulative statistics into time-based operational signals. PostgreSQL exposes counters such as commits, rollbacks, tuples, blocks, sessions, deadlocks, temporary files and checksum failures. A single reading shows a cumulative value; two readings taken at known times allow us to calculate what happened during that interval.

```

pg_stat_database
|
| snapshot every minute
v
monitor.database_snapshots
|
| current - previous
v
monitor.database_delta
|
+-- transaction throughput
+-- rollback rate
+-- cache hit ratio
+-- database activity
+-- session growth
+-- abandoned / fatal / killed sessions
+-- temp-file activity
+-- tuple activity
+-- deadlocks
+-- checksum failures
+-- workload ranking

```

PostgreSQL statistics are cumulative and can be reset. Delta logic must therefore detect a statistics reset and must never treat a negative difference as valid workload.

2. Scope and Important Corrections to the Picture

- Transaction throughput, rollbacks, cache activity, tuple activity, temporary files, deadlocks, checksum failures and session counters can be derived from `pg_stat_database` deltas.
- “Database I/O latency” is not accurately derived from block counters alone. PostgreSQL 18 exposes I/O timing through `pg_stat_database` and `pg_stat_io` when the relevant timing collection is enabled.
- “Parallel worker pressure” is not a `pg_stat_database` metric. It should be observed from `pg_stat_activity` and the configured worker limits.
- Current active sessions are a gauge from `pg_stat_activity`; session growth over an interval can use `pg_stat_database.sessions`.
- Cache hit ratio is a PostgreSQL shared-buffer hit ratio, not an operating-system filesystem-cache hit ratio.
- Checksum failures are meaningful only when data checksums are enabled; PostgreSQL 18 enables checksums by default for newly initialized clusters unless explicitly disabled.

3. Prerequisites

- PostgreSQL 18.x.
- A dedicated monitoring database or a chosen administrative database.
- A monitoring owner role with permission to create objects in a dedicated schema.
- A scheduler that can execute one SQL statement every minute (pg_cron if separately installed, systemd timer, cron, Kubernetes CronJob, enterprise scheduler, etc.).
- Sufficient retention storage for snapshots.
- track_counts enabled (default behavior).
- For I/O latency metrics, evaluate and enable track_io_timing if acceptable for the platform.

Do not reset statistics casually: Functions that reset PostgreSQL statistics break continuity of delta calculations. Record stats_reset and treat the first sample after a reset as a new baseline.

4. Monitoring Schema Installation

```
CREATE SCHEMA IF NOT EXISTS monitor;
```

```
REVOKE ALL ON SCHEMA monitor FROM PUBLIC;
```

```
CREATE TABLE monitor.database_snapshots (
  snapshot_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  captured_at      timestampz NOT NULL DEFAULT clock_timestamp(),
  datid            oid NOT NULL,
  datname          name,
  numbackends      integer NOT NULL,
  xact_commit      bigint NOT NULL,
  xact_rollback    bigint NOT NULL,
  blks_read        bigint NOT NULL,
  blks_hit         bigint NOT NULL,
  tup_returned     bigint NOT NULL,
  tup_fetched      bigint NOT NULL,
  tup_inserted     bigint NOT NULL,
  tup_updated      bigint NOT NULL,
  tup_deleted      bigint NOT NULL,
  conflicts        bigint NOT NULL,
  temp_files       bigint NOT NULL,
  temp_bytes       bigint NOT NULL,
  deadlocks        bigint NOT NULL,
  checksum_failures    bigint,
  checksum_last_failure timestampz,
  blk_read_time    double precision NOT NULL,
  blk_write_time   double precision NOT NULL,
  session_time     double precision NOT NULL,
  active_time      double precision NOT NULL,
  idle_in_transaction_time double precision NOT NULL,
  sessions         bigint NOT NULL,
```

```

sessions_abandoned    bigint NOT NULL,
sessions_fatal         bigint NOT NULL,
sessions_killed        bigint NOT NULL,
stats_reset            timestamptz
);

CREATE INDEX database_snapshots_datid_time_idx
ON monitor.database_snapshots (datid, captured_at DESC);

```

5. Snapshot Collection Function

```

CREATE OR REPLACE FUNCTION monitor.capture_database_snapshot()
RETURNS void
LANGUAGE sql
SECURITY INVOKER
SET search_path = pg_catalog, monitor
AS $$
    INSERT INTO monitor.database_snapshots (
        datid, datname, numbackends,
        xact_commit, xact_rollback,
        blks_read, blks_hit,
        tup_returned, tup_fetched,
        tup_inserted, tup_updated, tup_deleted,
        conflicts, temp_files, temp_bytes, deadlocks,
        checksum_failures, checksum_last_failure,
        blk_read_time, blk_write_time,
        session_time, active_time, idle_in_transaction_time,
        sessions, sessions_abandoned, sessions_fatal, sessions_killed,
        stats_reset
    )
    SELECT
        datid, datname, numbackends,
        xact_commit, xact_rollback,
        blks_read, blks_hit,
        tup_returned, tup_fetched,
        tup_inserted, tup_updated, tup_deleted,
        conflicts, temp_files, temp_bytes, deadlocks,
        checksum_failures, checksum_last_failure,
        blk_read_time, blk_write_time,
        session_time, active_time, idle_in_transaction_time,
        sessions, sessions_abandoned, sessions_fatal, sessions_killed,
        stats_reset
    FROM pg_stat_database
    WHERE datname IS NOT NULL;
$$;

```

```
SELECT monitor.capture_database_snapshot();
```

6. Delta View

The view below compares each sample with the previous sample for the same database. It rejects intervals that cross a statistics reset and protects ratio calculations from division by zero.

```
CREATE OR REPLACE VIEW monitor.database_delta AS
WITH s AS (
  SELECT
    d.*,
    lag(snapshot_id)      OVER w AS prev_snapshot_id,
    lag(captured_at)      OVER w AS prev_captured_at,
    lag(xact_commit)      OVER w AS p_xact_commit,
    lag(xact_rollback)    OVER w AS p_xact_rollback,
    lag(blks_read)        OVER w AS p_blks_read,
    lag(blks_hit)         OVER w AS p_blks_hit,
    lag(tup_returned)     OVER w AS p_tup_returned,
    lag(tup_fetched)      OVER w AS p_tup_fetched,
    lag(tup_inserted)     OVER w AS p_tup_inserted,
    lag(tup_updated)      OVER w AS p_tup_updated,
    lag(tup_deleted)      OVER w AS p_tup_deleted,
    lag(temp_files)       OVER w AS p_temp_files,
    lag(temp_bytes)       OVER w AS p_temp_bytes,
    lag(deadlocks)        OVER w AS p_deadlocks,
    lag(checksum_failures) OVER w AS p_checksum_failures,
    lag(blk_read_time)    OVER w AS p_blk_read_time,
    lag(blk_write_time)   OVER w AS p_blk_write_time,
    lag(sessions)         OVER w AS p_sessions,
    lag(sessions_abandoned) OVER w AS p_sessions_abandoned,
    lag(sessions_fatal)   OVER w AS p_sessions_fatal,
    lag(sessions_killed)  OVER w AS p_sessions_killed,
    lag(stats_reset)      OVER w AS p_stats_reset
  FROM monitor.database_snapshots d
  WINDOW w AS (PARTITION BY datid ORDER BY captured_at, snapshot_id)
)
SELECT
  snapshot_id, prev_snapshot_id, datid, datname,
  captured_at, prev_captured_at,
  extract(epoch FROM captured_at - prev_captured_at) AS interval_seconds,
  numbackends,
  xact_commit - p_xact_commit AS commits,
  xact_rollback - p_xact_rollback AS rollbacks,
  blks_read - p_blks_read AS blocks_read,
  blks_hit - p_blks_hit AS blocks_hit,
```

```

tup_returned - p_tup_returned AS tuples_returned,
tup_fetched - p_tup_fetched AS tuples_fetched,
tup_inserted - p_tup_inserted AS tuples_inserted,
tup_updated - p_tup_updated AS tuples_updated,
tup_deleted - p_tup_deleted AS tuples_deleted,
temp_files - p_temp_files AS temp_files,
temp_bytes - p_temp_bytes AS temp_bytes,
deadlocks - p_deadlocks AS deadlocks,
checksum_failures - p_checksum_failures AS checksum_failures,
blk_read_time - p_blk_read_time AS read_ms,
blk_write_time - p_blk_write_time AS write_ms,
sessions - p_sessions AS sessions_started,
sessions_abandoned - p_sessions_abandoned AS sessions_abandoned,
sessions_fatal - p_sessions_fatal AS sessions_fatal,
sessions_killed - p_sessions_killed AS sessions_killed,
stats_reset
FROM s
WHERE prev_snapshot_id IS NOT NULL
AND stats_reset IS NOT DISTINCT FROM p_stats_reset
AND captured_at > prev_captured_at
AND xact_commit >= p_xact_commit
AND xact_rollback >= p_xact_rollback;

```

7. Production Monitoring Queries

7.1 Transaction Throughput

Commits and rollbacks per second.

```

SELECT datname, captured_at,
       round((commits + rollbacks)::numeric / NULLIF(interval_seconds,0), 2) AS tx_per_sec
FROM monitor.database_delta
ORDER BY captured_at DESC, tx_per_sec DESC;

```

7.2 Rollback Detector

A rising rollback percentage can indicate application errors, contention, explicit rollbacks or failed transactions.

```

SELECT datname, captured_at, commits, rollbacks,
       round(100.0 * rollbacks / NULLIF(commits + rollbacks,0), 2) AS rollback_pct
FROM monitor.database_delta
WHERE commits + rollbacks > 0
ORDER BY captured_at DESC;

```

7.3 Cache Hit Ratio

This is the ratio of shared-buffer hits to hits plus reads requested through PostgreSQL's buffer manager.

```

SELECT datname, captured_at, blocks_hit, blocks_read,
       round(100.0 * blocks_hit / NULLIF(blocks_hit + blocks_read,0), 2) AS buffer_hit_pct

```



```
FROM monitor.database_delta
WHERE blocks_hit + blocks_read > 0
ORDER BY captured_at DESC;
```

7.4 Database Activity

Use current backend count as a gauge and transaction rate as interval activity.

```
SELECT datname, captured_at, numbackends,
       round((commits + rollbacks)::numeric / NULLIF(interval_seconds,0),2) AS tx_per_sec
FROM monitor.database_delta
ORDER BY captured_at DESC;
```

7.5 Session Growth

New sessions during each interval.

```
SELECT datname, captured_at, sessions_started,
       round(sessions_started::numeric / NULLIF(interval_seconds,0),2) AS new_sessions_per_sec
FROM monitor.database_delta
ORDER BY captured_at DESC;
```

7.6 Abandoned / Fatal / Killed Sessions

PostgreSQL 18 exposes cumulative session outcome counters in pg_stat_database.

```
SELECT datname, captured_at,
       sessions_abandoned, sessions_fatal, sessions_killed
FROM monitor.database_delta
WHERE sessions_abandoned > 0 OR sessions_fatal > 0 OR sessions_killed > 0
ORDER BY captured_at DESC;
```

7.7 Temporary File Detector

Temporary files and bytes are cumulative database counters; deltas show interval spill activity.

```
SELECT datname, captured_at, temp_files,
       pg_size_pretty(temp_bytes) AS temp_written
FROM monitor.database_delta
WHERE temp_files > 0 OR temp_bytes > 0
ORDER BY temp_bytes DESC;
```

7.8 Tuple Activity

Logical tuple counters help show database activity, but they are not a direct physical disk-I/O measurement.

```
SELECT datname, captured_at,
       tuples_returned, tuples_fetched,
       tuples_inserted, tuples_updated, tuples_deleted
FROM monitor.database_delta
ORDER BY captured_at DESC;
```

7.9 Deadlock Detector

Any positive delta means PostgreSQL detected one or more deadlocks in the interval.

```
SELECT datname, captured_at, deadlocks
FROM monitor.database_delta
WHERE deadlocks > 0
ORDER BY captured_at DESC;
```

7.10 Checksum Failure Detector

Use only when data checksums are enabled. A new failure requires immediate investigation.

```
SHOW data_checksums;

SELECT datname, captured_at, checksum_failures
FROM monitor.database_delta
WHERE checksum_failures > 0
ORDER BY captured_at DESC;
```

7.11 Database Workload Ranking

A simple activity score can rank databases by transaction volume. Treat it as a ranking aid, not a universal load metric.

```
SELECT datname,
       sum(commits + rollbacks) AS transactions,
       sum(temp_bytes) AS temp_bytes,
       sum(tuples_inserted + tuples_updated + tuples_deleted) AS row_changes
FROM monitor.database_delta
WHERE captured_at >= now() - interval '1 hour'
GROUP BY datname
ORDER BY transactions DESC;
```

8. Accurate I/O Latency Monitoring

For database-level timing, `pg_stat_database` exposes `blk_read_time` and `blk_write_time` when `track_io_timing` is enabled. PostgreSQL 18 also provides `pg_stat_io` with cluster-wide I/O statistics by backend type, object and context, including byte counters. PostgreSQL documentation recommends combining PostgreSQL I/O statistics with operating-system tools because PostgreSQL cannot tell whether a read was satisfied from the kernel page cache or physical storage.

```
SHOW track_io_timing;

-- Enable only after evaluating timing overhead for your platform:
ALTER SYSTEM SET track_io_timing = on;
-- Reload configuration using your normal controlled change procedure.

SELECT backend_type, object, context,
       reads, read_bytes, read_time,
       writes, write_bytes, write_time,
       fsyncs, fsync_time
FROM pg_stat_io
ORDER BY COALESCE(read_time,0) + COALESCE(write_time,0) DESC NULLS LAST;
```

Interpretation: Do not label blk_read_time / blocks_read as physical-disk latency. It is PostgreSQL I/O timing for operations PostgreSQL performed; the OS page cache can affect the physical storage path.

9. Parallel Worker Pressure

Parallel worker pressure is a live cluster-level observation, not a pg_stat_database delta. Compare active parallel-worker backends with max_parallel_workers and max_worker_processes, and inspect parallel query configuration.

```
SHOW max_worker_processes;
SHOW max_parallel_workers;
SHOW max_parallel_workers_per_gather;

SELECT backend_type, count(*) AS processes
FROM pg_stat_activity
GROUP BY backend_type
ORDER BY processes DESC;

SELECT count(*) AS active_parallel_workers
FROM pg_stat_activity
WHERE backend_type = 'parallel worker';
```

10. Current Session and Wait Monitoring

```
SELECT
  pid, datname, username, application_name,
  client_addr, state, backend_type,
  wait_event_type, wait_event,
  xact_start, query_start,
  now() - query_start AS query_age,
  left(query, 200) AS query
FROM pg_stat_activity
WHERE pid <> pg_backend_pid()
ORDER BY query_start NULLS LAST;
```

Use this live view for current waits, long-running activity and backend types. The snapshot framework complements it; it does not replace pg_stat_activity.

11. Optional SQL Workload Layer: pg_stat_statements

Database-wide counters tell you that load increased; pg_stat_statements helps identify which normalized SQL statements consumed execution resources. It is a supplied PostgreSQL module but requires shared_preload_libraries and a restart when first enabled.

```
-- postgresql.conf / controlled configuration:
shared_preload_libraries = 'pg_stat_statements'
compute_query_id = auto

-- After restart, in each database to be analyzed:
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
```

```
SELECT queryid, calls, total_exec_time, mean_exec_time, rows,
       shared_blks_hit, shared_blks_read,
       temp_blks_read, temp_blks_written,
       left(query, 250) AS query
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 20;
```

12. Scheduling the Snapshot

The core PostgreSQL script is scheduler-neutral. Run the following exactly once per intended interval:

```
SELECT monitor.capture_database_snapshot();
```

- Linux cron/systemd: invoke psql using a protected service account and .pgpass or an approved secret mechanism.
- Kubernetes: use a CronJob only if that operational model is approved; avoid embedding passwords in manifests.
- Enterprise scheduler: run the same SELECT statement and alert on non-zero exit status.
- If pg_cron is used, treat it as an extension with its own installation/version lifecycle; it is not PostgreSQL core.

13. Retention and Housekeeping

```
-- Example: retain 30 days.
DELETE FROM monitor.database_snapshots
WHERE captured_at < now() - interval '30 days';

-- Review table size:
SELECT pg_size_pretty(pg_total_relation_size('monitor.database_snapshots'))
       AS snapshot_table_size;
```

Choose retention from operational and audit requirements. For large environments, consider time-based partitioning instead of an ever-growing single table.

14. Suggested Alert Conditions

Signal	Example Condition	Meaning
Deadlocks	delta > 0	Investigate conflicting transaction/lock patterns.
Checksum failures	delta > 0	Possible page corruption; urgent investigation.
Fatal/killed/abandoned sessions	delta > baseline	Connection/server instability or abnormal termination.
Rollback %	sustained above workload baseline	Application failures, contention, explicit rollbacks.
Temp bytes	sharp sustained rise	Sort/hash spill or large temporary workload.
New sessions/sec	sharp rise	Connection churn or pool issue.

Buffer hit %	material fall from baseline	Workload/cache behavior changed; investigate with I/O data.
Parallel workers	near configured capacity with demand	Potential worker saturation; correlate with workload.
I/O timing	material rise from baseline	I/O path may be slower; correlate with OS/storage telemetry.
Threshold policy: Do not use universal magic thresholds. Build baselines for your workload and alert on sustained deviation, SLO impact, or known failure indicators.		

15. Validation Procedure

1. Run the installation DDL in a non-production environment.
2. Capture one snapshot and confirm one row per database plus expected fields.
3. Wait at least one collection interval and capture a second snapshot.
4. Query `monitor.database_delta` and confirm non-negative differences.
5. Generate controlled transactions and verify transaction deltas.
6. Create a controlled rollback and verify rollback increment.
7. Create controlled temporary-file activity only in a lab and verify temp counters.
8. Confirm `SHOW data_checksums` before interpreting checksum counters.
9. Confirm `SHOW track_io_timing` before interpreting I/O timing deltas.
10. Reset statistics only in a lab, capture again, and confirm the reset boundary is excluded.
11. Validate permissions using the real monitoring service account.
12. Load-test the one-minute collection overhead before production rollout.

16. Troubleshooting

Symptom	Check	Action
No delta rows	Need at least two samples; check <code>stats_reset</code>	Capture another sample; inspect reset boundary.
Negative counters	Statistics reset or bad source history	Do not calculate across reset; validate collector.
I/O times stay zero	<code>SHOW track_io_timing</code>	Enable only through controlled change if required.
Checksum failures null/zero	<code>SHOW data_checksums</code>	Confirm checksums are enabled and understand NULL semantics.
Large snapshot table	Retention and collection frequency	Purge/partition per policy.
Permission error	Schema/table/function grants	Grant least privilege to monitoring role.
Parallel pressure missing	Wrong source	Use <code>pg_stat_activity</code> and worker settings, not <code>pg_stat_database</code> .
High cache hit but slow DB	Cache ratio is not end-to-end latency	Inspect waits, <code>pg_stat_io</code> , SQL, OS and storage.

17. Security and Least Privilege

- Keep monitoring objects in a dedicated schema.

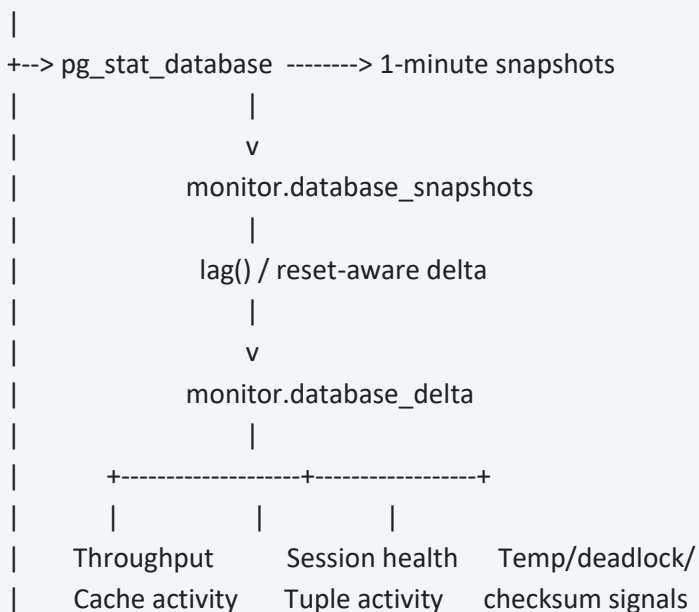
- Do not grant application roles write access to monitoring tables.
- Use a dedicated collector identity.
- Protect credentials outside scripts and source repositories.
- Restrict access to query text from `pg_stat_activity` and `pg_stat_statements` because it can expose sensitive SQL literals.
- Review PostgreSQL predefined monitoring roles where appropriate instead of granting superuser.

18. Production Rollout Checklist

- ☐ PostgreSQL 18 minor release confirmed and documentation reviewed.
- ☐ Monitoring schema deployed from version-controlled SQL.
- ☐ Collector role follows least privilege.
- ☐ One-minute schedule tested under representative load.
- ☐ `stats_reset` handling validated.
- ☐ `track_io_timing` decision documented.
- ☐ Data-checksum state documented.
- ☐ Retention and purge job configured.
- ☐ Monitoring table growth alert configured.
- ☐ Deadlock/checksum/session anomaly alerts configured.
- ☐ OS/storage telemetry integrated for I/O correlation.
- ☐ `pg_stat_statements` enabled only if approved and needed.
- ☐ Runbook owner and escalation path assigned.
- ☐ Monitoring queries tested after PostgreSQL upgrades.

19. Complete Architecture Summary

PostgreSQL cumulative statistics



```

|
+--> pg_stat_activity -----> live sessions / waits / parallel workers
|
+--> pg_stat_io -----> I/O operations, bytes and timing
|
+--> pg_stat_statements ----> optional SQL workload attribution
|
+--> OS / storage tools ----> physical-system correlation

```

20. Official PostgreSQL References

PostgreSQL 18 — Monitoring Database Activity

<https://www.postgresql.org/docs/18/monitoring.html>

PostgreSQL 18 — Cumulative Statistics System

<https://www.postgresql.org/docs/18/monitoring-stats.html>

PostgreSQL 18 — Run-time Statistics

<https://www.postgresql.org/docs/18/runtime-config-statistics.html>

PostgreSQL 18 — Resource Consumption / Worker Processes

<https://www.postgresql.org/docs/18/runtime-config-resource.html>

PostgreSQL 18 — Data Checksums

<https://www.postgresql.org/docs/18/checksums.html>

PostgreSQL 18 — pg_stat_statements

<https://www.postgresql.org/docs/18/pgstatstatements.html>

PostgreSQL Documentation

<https://www.postgresql.org/docs/>

Appendix A — Deployment SQL Order

- 1. Create monitor schema.
- 2. Create monitor.database_snapshots.
- 3. Create supporting index.
- 4. Create monitor.capture_database_snapshot().
- 5. Create monitor.database_delta.
- 6. Execute first capture.
- 7. Execute second capture after the chosen interval.
- 8. Validate delta queries.
- 9. Configure external scheduling.
- 10. Configure retention and alerting.

Appendix B — DBA Interpretation Rules

- Always distinguish counters from gauges.
- Always calculate counters inside the same stats_reset epoch.
- Never infer physical storage latency from buffer hit ratio.
- Never infer query-level root cause from database-level counters alone.
- Correlate PostgreSQL metrics with waits, SQL statistics, OS CPU/memory, filesystem and storage metrics.

- Treat checksum failures and deadlocks as event signals, not normal performance percentages.
- Use workload-specific baselines rather than generic thresholds.